

Universidade Federal de Uberlândia

Aula 14: Busca em árvore Monte Carlo II

AlphaZero

PUCT • Auto-jogo • Rede política–valor • AlphaTensor e AlphaDev • Raciocínio metanível

Prof. Marcelo Keese Albertini
Universidade Federal de Uberlândia
adaptado de Emma Brunskill, CS234 (Stanford)
2026



1. **Planejar agora, em vez de resolver tudo** — por que faz sentido gastar computação local para decidir *uma* jogada, e o que a busca em árvore Monte Carlo (MCTS) faz com esse orçamento.
2. **AlphaZero** — o que acontece quando a rollout aleatória é substituída por uma rede que devolve *política* e *valor*: a regra PUCT, as quatro fases, a política da raiz.
3. **Auto-jogo** — o laço que transforma busca em dados de treino, e por que o adversário ser uma cópia de si mesmo é uma vantagem, não um acaso.
4. **A evidência** — quanto vem da arquitetura, quanto vem da busca, quanto vem dos dados humanos. Os números do artigo.
5. **Além do tabuleiro** — AlphaTensor, AlphaDev, MuZero: descoberta científica reescrita como problema de busca.
6. **Um olhar crítico** — o UCT é mesmo o algoritmo certo para gastar simulações? Aqui a resposta é mais interessante do que parece.

Onde isto se encaixa no curso

Até aqui, quase tudo foi sobre calcular uma política **para todo o espaço de estados**: iteração de valor, Q -learning, DQN, gradientes de política, PPO. O objeto produzido é uma função $\pi : \mathcal{S} \rightarrow \mathcal{A}$.

Hoje o objeto é outro: dado o estado *atual* s_t , gastar computação para escolher **uma** ação boa — e só ela. Depois, repetir.

As duas visões não competem: AlphaZero usa uma para melhorar a outra.

Intuição

Uma política global é uma resposta *amortizada*: paga-se caro uma vez e consulta-se barato para sempre. O planejamento local é uma resposta *sob demanda*: nada é pago antecipadamente, mas cada decisão custa. Quando $|\mathcal{S}|$ é astronômico, só a segunda é viável — e a primeira vira uma *aproximação* que a segunda corrige na hora.

SEÇÃO 1

Planejar agora, não resolver tudo

01

Busca Monte Carlo simples

Um passo de melhoria de política, e nada mais

Dados um modelo (simulador) $\mathcal{M}$ e uma política de simulação π :

- ▶ Para cada ação $a \in \mathcal{A}$, simule K episódios a partir do estado real s_t :

$$\{s_t, a, r_{t+1}^k, s_{t+1}^k, \dots, s_T^k\}_{k=1}^K \sim \mathcal{M}, \pi$$

- ▶ Avalie cada ação pelo retorno médio:

$$Q(s_t, a) := \frac{1}{K} \sum_{k=1}^K G_t^k \xrightarrow{K \rightarrow \infty} Q^\pi(s_t, a)$$

- ▶ Escolha a ação real $a_t = \arg \max_{a \in \mathcal{A}} Q(s_t, a)$.

Atenção

Isto é exatamente **um passo** de melhoria de política sobre π : o resultado é guloso-em-relação-a- Q^π , não π^* . Se π for ruim, a estimativa que orienta a decisão também é.

Podemos fazer melhor: a árvore expectimax

Com o modelo em mãos, dá para calcular $Q^*(s_t, a)$ *exatamente* — basta expandir a árvore de decisão completa a partir de s_t e retropropagar

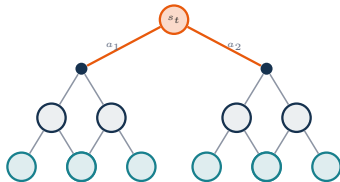
$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) \max_{a'} Q^*(s', a').$$

Definição — Busca à frente

Não é preciso resolver o MDP inteiro — apenas o *sub-MDP* que começa agora e vai até o horizonte H .

Atenção

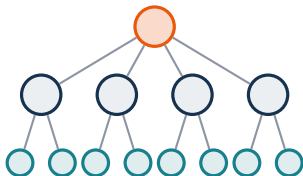
O tamanho da árvore cresce como $(|S||\mathcal{A}|)^H$. No Go: fator de ramificação ≈ 250 , partida com ≈ 150 jogadas. Uma árvore de $250^{150} \approx 10^{360}$ folhas.



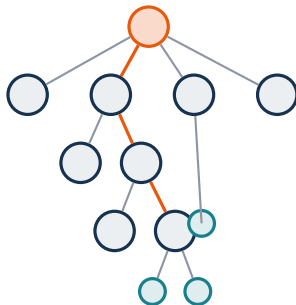
Busca em árvore Monte Carlo: crescer a árvore onde importa

A resposta da MCTS ao $(|\mathcal{S}||\mathcal{A}|)^H$ é não construir a árvore toda. Constrói-se uma árvore **assimétrica**, aprofundada só nas regiões que as simulações indicaram valer a pena.

expectimax: uniforme e completa

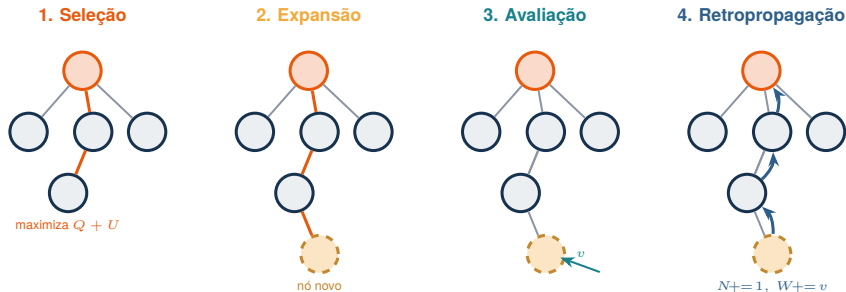


MCTS: seletiva e profunda onde promete



A árvore da direita é o *produto* do algoritmo, não sua entrada: cada simulação a torna um pouco mais informativa exatamente onde a decisão é difícil.

As quatro fases de uma simulação



Repita K vezes. Ao final, a ação real é escolhida a partir das estatísticas da raiz. **Todo o desenho do algoritmo está em duas escolhas:** como selecionar na fase 1, e de onde vem o v da fase 3.

UCT: cada nó da árvore é um bandit

A fase de seleção precisa de uma regra. O UCT (Kocsis e Szepesvári, 2006) empresta a ideia dos bandits: em cada nó i onde há escolha, trate as ações como braços e mantenha uma cota superior de confiança.

Regra de seleção do UCT

$$Q(s, a, i) = \underbrace{\frac{1}{N(i, a)} \sum_{k=1}^{N(i, a)} G_k(i, a)}_{\text{média dos retornos simulados}} + \underbrace{c \sqrt{\frac{\log N(i)}{N(i, a)}}}_{\text{bônus de otimismo}}, \quad a_{ik} = \arg \max_a Q(s, a, i)$$

- ▶ $N(i, a)$: quantas simulações passaram por a no nó i ; $G_k(i, a)$: o k -ésimo retorno observado a partir dali.
- ▶ Consequência importante: **a política de simulação muda a cada episódio**. A árvore não é percorrida por uma política fixa — ela é percorrida por uma política que se adapta ao que já foi visto.
- ▶ Guarde esta observação: a média $\bar{G}$ é uma média sobre uma política *não estacionária*. Voltaremos a isso na última seção.

Por que a MCTS funciona tão bem

- ▶ **Busca seletiva pelo melhor primeiro.** O orçamento vai para onde a decisão é apertada, não para onde a árvore é larga.
- ▶ **Avaliação dinâmica de estados.** Não há função de avaliação fixa: o valor de um nó é construído pelas simulações que passam por ele.
- ▶ **Amostragem quebra a maldição da dimensionalidade.** O custo deixa de depender de $|S|$ e passa a depender do orçamento de simulações.
- ▶ **Basta um modelo “caixa-preta”.** Nada de $P(s' | s, a)$ explícito — só a capacidade de *amostrar* transições.
- ▶ **Barato, *anytime* e paralelizável.** Interrompa quando o relógio acabar; a resposta atual já é utilizável.

Intuição

“Anytime” é uma propriedade rara e valiosa. A iteração de valor não entrega nada útil até convergir razoavelmente; a MCTS entrega uma resposta defensável após a primeira simulação e vai refinando. Em xadrez, Go e controle em tempo real, isso é a diferença entre usável e inútil.

Atenção

O custo é escondido no orçamento: cada decisão real exige milhares de simulações. A MCTS troca memória e pré-computação por *tempo de decisão*.

Estudo de caso: o jogo de Go

- ▶ 2 500 anos de idade; o jogo de tabuleiro clássico mais difícil.
- ▶ Desafio histórico da IA — John McCarthy o citava como marco.
- ▶ A busca em árvore tradicional (alfa-beta com função de avaliação manual) **fracassou** no Go, ao contrário do xadrez.

	Xadrez	Go
Ramificação típica	≈ 35	≈ 250
Duração da partida	≈ 80	≈ 150
Posições legais	$\approx 10^{47}$	$\approx 10^{170}$
Avaliação manual boa?	sim	não

Teste sua compreensão

Jogar Go é aprender a decidir num mundo cuja **dinâmica** e cujo **modelo de recompensa** são desconhecidos? *Pergunta para discussão conduzida em aula.*

Estudo de caso: o jogo de Go

- ▶ 2 500 anos de idade; o jogo de tabuleiro clássico mais difícil.
- ▶ Desafio histórico da IA — John McCarthy o citava como marco.
- ▶ A busca em árvore tradicional (alfa-beta com função de avaliação manual) **fracassou** no Go, ao contrário do xadrez.

	Xadrez	Go
Ramificação típica	≈ 35	≈ 250
Duração da partida	≈ 80	≈ 150
Posições legais	$\approx 10^{47}$	$\approx 10^{170}$
Avaliação manual boa?	sim	não

Teste sua compreensão

Jogar Go é aprender a decidir num mundo cuja **dinâmica** e cujo **modelo de recompensa** são desconhecidos? *Pergunta para discussão conduzida em aula.*

Resposta

Não. As regras do Go são conhecidas e determinísticas; a recompensa (vitória/derrota) é conhecida e verificável ao final. O que falta não é o *modelo* — é a capacidade de *usá-lo*: $|S| \approx 10^{170}$ e $H \approx 150$. AlphaZero é, antes de tudo, **planejamento** com heurísticas aprendidas.

SEÇÃO 2

AlphaZero: a busca guiada por uma rede

02

De AlphaGo a AlphaZero

Três sistemas, três graus de conhecimento humano embutido

	AlphaGo (2016)	AlphaGo Zero (2017)	AlphaZero (2018)
Partidas humanas	sim (rede supervisionada)	não	não
Redes	4 separadas	1 rede, 2 cabeças	1 rede, 2 cabeças
Arquitetura	convolucional	residual	residual
Avaliação da folha	rollout + rede de valor	só a rede de valor	só a rede de valor
Atributos manuais	vários	só as regras	só as regras
Domínios	Go	Go	Go, xadrez, shogi

Intuição

A trajetória é uma sequência de *remoções*: cada versão tirou uma muleta humana e ficou mais forte. O detalhe contraintuitivo é que retirar a rollout — a parte “Monte Carlo” da busca em árvore Monte Carlo — *melhorou* o sistema. A otimização bayesiana dos hiperparâmetros do AlphaGo já vinha deslocando o peso da mistura para a rede de valor a cada ciclo, até a rollout ser abandonada de vez (Chen et al., 2018).

A rede: uma função, duas saídas

Uma única rede residual f_θ recebe a posição s (o tabuleiro cru, mais o histórico recente e de quem é a vez) e devolve

$$f_\theta(s) = (p_\theta(s), v_\theta(s)), \quad p_\theta(s) \in \Delta(\mathcal{A}), \quad v_\theta(s) \in [-1, 1].$$

- ▶ $p_\theta(s)$ — **cabeça de política**: uma distribuição sobre as jogadas legais. Papel na busca: *prior* que diz por onde começar a olhar.
- ▶ $v_\theta(s)$ — **cabeça de valor**: estimativa do resultado final do ponto de vista de quem joga agora (+1 vitória, -1 derrota). Papel na busca: substitui a rollout.

Definição — Corpo compartilhado

As duas cabeças partilham a torre residual. “Onde estou ganhando” e “o que jogar aqui” exigem as mesmas representações — reconhecer grupos vivos e mortos, influência, forma.

Uma passagem ≈ 100 rollouts

No AlphaGo de 2016, uma única passagem direta de v_θ atingiu erro quadrático médio comparável à média de 100 rollouts com a política de RL — a um custo de computação ordens de grandeza menor.

PUCT: a regra de seleção do AlphaZero

Em cada nó da árvore, mantêm-se por ação: $N(s, a)$ (visitas), $W(s, a)$ (soma dos valores retropropagados), $Q(s, a) = W/N$ e o prior $P(s, a) = p_\theta(s)_a$. A ação simulada é

PUCT — *Predictor* + UCT

$$a_t = \arg \max_a \left[\underbrace{Q(s, a)}_{\text{exploração aprendida}} + \underbrace{c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}}_{U(s, a) : \text{bônus guiado pelo prior}} \right]$$

- ▶ Enquanto $N(s, a)$ é pequeno, o termo U domina e a busca segue o **prior da rede**: começa-se olhando o que a política acha plausível.
- ▶ À medida que as visitas se acumulam, $U \rightarrow 0$ e Q assume: a busca **corrige** a rede com evidência obtida por lookahead.
- ▶ c_{puct} regula a data dessa transferência de poder.

UCT e PUCT lado a lado

	UCT	PUCT (AlphaZero)
Bônus	$c \sqrt{\frac{\log N(s)}{N(s, a)}}$	$c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)}$
Decaimento em $N(s, a)$	$N(s, a)^{-1/2}$	$N(s, a)^{-1}$ (mais agressivo)
Crescimento em $N(s)$	$\sqrt{\log N(s)}$	$\sqrt{N(s)}$ (muito mais forte)
Ação nunca visitada	bônus infinito	bônus finito, $\propto P(s, a)$
Origem	desigualdade de Hoeffding	heurística
Avaliação da folha	rollout até o fim	v_θ , sem rollout

Atenção

O PUCT **não** é uma cota de confiança. Ele perdeu a justificativa estatística do UCT e ganhou outra, mais útil: *as ações com prior baixo podem nunca ser visitadas*. Em Go, com ≈ 250 jogadas legais e 1 600 simulações, isso é o que torna a busca viável — e é também o que exige o ruído de Dirichlet que veremos adiante.

A simulação, do começo ao fim

1. Seleção. Da raiz, desça escolhendo $\arg \max_a [Q + U]$ até chegar a um nó s_L que ainda não está na árvore.

2. Expansão e avaliação. Uma única chamada $f_\theta(s_L) = (p_\theta, v_\theta)$. O nó entra na árvore com $N = W = Q = 0$ e priors p_θ ; o valor $v = v_\theta(s_L)$ é a avaliação. **Não há rollout.**

3. Retropropagação. Para cada aresta (s, a) do caminho:

$$N += 1, \quad W += \pm v, \quad Q = W/N.$$

O sinal alterna com o jogador da vez.

Intuição

Trocar a rollout pela rede é a mesma troca de MC por TD. A rollout é um retorno Monte Carlo: não enviesado em relação à política de rollout, mas de variância enorme e enviesado em relação a π^* , porque a política de rollout é fraca. A rede de valor é um *bootstrap*: enviesado, de variância baixa, e o viés diminui com o treino.

Definição — Min-max implícito

Como v é sempre “do ponto de vista de quem joga”, alternar o sinal na subida faz a árvore se comportar como uma árvore min-max — sem que “minimizar” apareça em lugar algum do código.

O PUCT em câmera lenta

Um exemplo mínimo que cabe no quadro

Raiz com três jogadas. A rede erra: dá prior alto para a_1 , que é a pior das duas jogadas razoáveis. Os valores das folhas são $v_\theta = (0,10; 0,55; -0,40)$ e $c_{\text{puct}} = 1,5$.

k	$Q(s, a)$			$U(s, a)$			$Q + U$			escolhe
	a_1	a_2	a_3	a_1	a_2	a_3	a_1	a_2	a_3	
1	0,00	0,00	0,00	0,90	0,45	0,15	0,90	0,45	0,15	a_1
2	0,10	0,00	0,00	0,45	0,45	0,15	0,55	0,45	0,15	a_1
3	0,10	0,00	0,00	0,42	0,64	0,21	0,52	0,64	0,21	a_2
4	0,10	0,55	0,00	0,52	0,39	0,26	0,62	0,94	0,26	a_2
5	0,10	0,55	0,00	0,60	0,30	0,30	0,70	0,85	0,30	a_2
6	0,10	0,55	0,00	0,67	0,25	0,34	0,77	0,80	0,34	a_2
7	0,10	0,55	0,00	0,74	0,22	0,37	0,84	0,77	0,37	a_1
8	0,10	0,55	0,00	0,60	0,24	0,40	0,70	0,79	0,40	a_2
após 16 simulações: $N = (5, 11, 0)$ • a_3 só recebe sua primeira visita em $k = 23$										

O que a tabela ensina

- ▶ $k = 1-2$: **a rede manda**. Nada foi avaliado ainda, então $Q \equiv 0$ e a escolha é puramente o prior.
- ▶ $k = 3$: **a evidência entra**. Assim que a_1 devolve 0,10 — abaixo do que o prior prometia — a busca migra.
- ▶ $k = 7$: **a busca volta a a_1** . O denominador $1 + N(s, a_2)$ cresceu; o sistema confere se não errou. Isso é o termo U fazendo seu trabalho.
- ▶ a_3 **fica de fora até $k = 23$** . Prior baixo somado a orçamento pequeno é igual a jogada nunca examinada.

Atenção

Quando o prior está muito errado, custa caro.

Com $P = (0,95; 0,04; 0,01)$ e o mesmo v_θ :

simulações	$N(a_1)$	$N(a_2)$	recomenda
50	33	17	a_1 (errado)
200	44	156	a_2
800	89	711	a_2

A busca *corrige* a rede, mas o preço é orçamento. Rede melhor $\Rightarrow$ menos simulações para a mesma decisão.

c_{puct} : quanto duvidar da própria rede

Mesmo exemplo, 400 simulações, variando só c_{puct} :

c_{puct}	$N(a_1)$	$N(a_2)$	$N(a_3)$	leitura
0,5	13	386	1	confia na rede; converge rápido, arrisca perder a jogada rara
1,0	25	373	2	
1,5	37	360	3	<i>faixa usada na prática</i>
3,0	71	324	5	
5,0	107	284	9	duvida da rede; gasta orçamento reexaminando o já descartado

Intuição

Não é um parâmetro cosmético. No AlphaGo, ajustar por otimização bayesiana os hiperparâmetros da MCTS — c_{puct} entre eles — rendeu **94 e 103 pontos de Elo** em dois ciclos de projeto antes da partida contra Lee Sedol. Cem pontos de Elo são cerca de 64% de vitórias contra a versão anterior.

Ruído de Dirichlet: garantir que tudo possa ser tentado

Se o prior é o que abre a porta e a porta de a_3 nunca abre, o sistema não consegue descobrir que a rede está errada sobre a_3 . Em **auto-jogo** (não em partida oficial), o AlphaZero perturba o prior *apenas na raiz*:

$$P(s_0, a) = (1 - \varepsilon) p_{\theta}(s_0)_a + \varepsilon \eta_a, \quad \eta \sim \text{Dir}(\alpha), \quad \varepsilon = 0,25.$$

Definição — Escolha de α

α é calibrado *inversamente* ao número típico de jogadas legais:

jogadas legais		α
Xadrez	≈ 35	0,30
Shogi	≈ 90	0,15
Go	≈ 250	0,03

α pequeno concentra o ruído em poucas jogadas sorteadas — exploração *profunda*, não diluição uniforme.

Intuição

O ruído fica só na raiz: a busca precisa *poder* tentar a jogada estranha, mas continua livre para descartá-la assim que Q mostrar que é ruim. É exploração que a evidência pode vetar — o oposto de ϵ -guloso, que força a ação aleatória.

Atenção

Em avaliação e partidas oficiais o ruído é removido. Ele é um mecanismo de *coleta de dados*, não de jogo.

A política da raiz: contagens de visita, não valores

Terminadas as K simulações, a política que o AlphaZero exporta na raiz é

$$\pi(a \mid s_0) = \frac{N(s_0, a)^{1/\tau}}{\sum_b N(s_0, b)^{1/\tau}}.$$

No exemplo anterior, com $N = (5, 11, 0)$:

τ	$\pi(a_1)$	$\pi(a_2)$	$\pi(a_3)$
1,00	0,312	0,688	0
0,50	0,171	0,829	0
0,25	0,041	0,959	0
$\rightarrow 0$	0	1	0

No auto-jogo: $\tau = 1$ nas 30 primeiras jogadas (diversidade de posições), depois $\tau \rightarrow 0$ (jogar para valer).

Teste sua compreensão

Por que exportar as *contagens de visita* em vez do $\arg \max_a Q(s_0, a)$, que parece a estimativa mais direta do que é bom? *Pergunta para discussão conduzida em aula.*

A política da raiz: contagens de visita, não valores

Terminadas as K simulações, a política que o AlphaZero exporta na raiz é

$$\pi(a \mid s_0) = \frac{N(s_0, a)^{1/\tau}}{\sum_b N(s_0, b)^{1/\tau}}.$$

No exemplo anterior, com $N = (5, 11, 0)$:

τ	$\pi(a_1)$	$\pi(a_2)$	$\pi(a_3)$
1,00	0,312	0,688	0
0,50	0,171	0,829	0
0,25	0,041	0,959	0
$\rightarrow 0$	0	1	0

No auto-jogo: $\tau = 1$ nas 30 primeiras jogadas (diversidade de posições), depois $\tau \rightarrow 0$ (jogar para valer).

Teste sua compreensão

Por que exportar as *contagens de visita* em vez do $\arg \max_a Q(s_0, a)$, que parece a estimativa mais direta do que é bom? *Pergunta para discussão conduzida em aula.*

Resposta

Porque N é mais robusto que Q : uma ação visitada 3 vezes pode ter Q alto por sorte. As contagens já incorporam a confiança — só recebe muitas visitas o que sobreviveu à comparação repetida.

Por que a busca melhora a política

A afirmação central de todo o sistema:

A MCTS como operador de melhoria de política

A política da raiz π é, tipicamente, mais forte do que a política p_θ que guiou a busca. A busca é um **operador de melhoria**: consome p_θ e v_θ , gasta computação, devolve algo melhor.

De onde vem a melhora. $p_\theta(s_0)$ julga a posição por reconhecimento de padrões; π julga a mesma posição *depois de olhar K continuações*. É informação que p_θ não tinha. **Consequência.** Se a busca melhora a política, treinar p_θ para imitar π é um passo de melhoria. Basta repetir: iteração de política, com a busca no lugar do passo guloso.

Intuição

Pensar devagar para treinar o pensar rápido.

A busca é deliberação, lenta e cara; a rede é intuição, rápida e barata. Anthony, Tian e Barber (2017) chamaram isso de *Expert Iteration*.

Atenção

“Tipicamente” não é “sempre”: se v_θ for sistematicamente enviesado numa região, mais busca amplifica o erro.

O que o PUCT está realmente otimizando

Grill et al., ICML 2020

As contagens de visita não são um subproduto arbitrário. Elas resolvem aproximadamente um problema de **otimização de política regularizada**:

$$\bar{\pi} \approx \arg \max_{y \in \Delta(\mathcal{A})} \left[\mathbf{q}^\top y - \lambda_N \text{KL}(\mathbf{p}_\theta \parallel y) \right], \quad \lambda_N = \frac{c_{\text{puct}} \sqrt{N}}{N + |\mathcal{A}|},$$

cuja solução é $\bar{\pi}_a = \lambda_N p_{\theta,a} / (\alpha - q_a)$, com α normalizando.

Verificando no exemplo do início ($P = (0,6; 0,3; 0,1)$, $q = (0,10; 0,55; -0,40)$, $c_{\text{puct}} = 1,5$):

N	visitas empíricas $N_a / (N + \mathcal{A})$			previsão da forma fechada		
	a_1	a_2	a_3	a_1	a_2	a_3
64	0,1940	0,7463	0,0149	0,2069	0,7755	0,0176
256	0,1120	0,8687	0,0077	0,1154	0,8752	0,0094
1600	0,0480	0,9464	0,0037	0,0486	0,9475	0,0039

Intuição

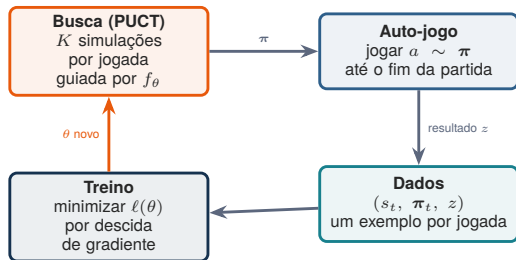
Como $\lambda_N \sim N^{-1/2} \rightarrow 0$, a regularização enfraquece com o orçamento: pouca busca $\Rightarrow$ fique perto do prior; muita busca $\Rightarrow$ vá para o $\arg \max$ de q . É a “proximidade regularizada” do TRPO e do PPO, agora *dentro* de um planejador.

SEÇÃO 3

Auto-jogo: o laço que fabrica os próprios dados

03

O laço completo



Nenhum humano no laço.

O único insumo externo são as **regras do jogo** — usadas pelo simulador para gerar sucessores e detectar o fim da partida.

Partidas por iteração	25 000
Simulações por jogada	1 600 ($\approx 0,4$ s por busca)
Temperatura	$\tau = 1$ nas 30 primeiras jogadas, depois $\tau \rightarrow 0$
Desistência	automática, mantendo falsos positivos abaixo de 5%

Os dois alvos, e por que só um é escasso

Cada jogada da partida gera um exemplo (s_t, π_t, z) , com $z \in \{-1, 0, +1\}$ o resultado *final* da partida visto de s_t .

A perda

$$\ell(\theta) = \underbrace{(z - v_\theta(s))^2}_{\text{regressão do valor}} - \underbrace{\pi^\top \log p_\theta(s)}_{\text{entropia cruzada: imitar a busca}} + \underbrace{c \|\theta\|^2}_{\text{regularização}}$$

- ▶ **Alvo de política: denso.** Uma partida de 200 jogadas dá 200 alvos π_t ricos — distribuições inteiras, não rótulos duros. É o passo de melhoria sendo destilado.
- ▶ **Alvo de valor: escasso.** Um único bit por partida, replicado em todas as posições dela. Alta correlação entre exemplos, sinal fraco por partida.

Atenção

Diz-se que “o Go tem recompensa esparsa”. Meia verdade: a *recompensa* é terminal, mas o *alvo de treino* da política é denso, porque a busca fabrica supervisão em cada posição. É um dos motivos pelos quais gradiente de política puro patinaria aqui.

Auto-jogo é um currículo automático

- ▶ **O gargalo é só computação.** Nenhum anotador, nenhum especialista, nenhum banco de partidas.
- ▶ **O adversário está sempre à altura.** Como é uma cópia de si mesmo, a taxa de vitórias fica em torno de 50% durante todo o treino.
- ▶ Um adversário na medida é a definição operacional de um **currículo**: a dificuldade sobe exatamente na velocidade da competência.

Teste sua compreensão

Em que isso ajuda o treino da política? E o que acontece com a densidade do sinal de recompensa? *Pergunta para discussão conduzida em aula.*

Auto-jogo é um currículo automático

- ▶ **O gargalo é só computação.** Nenhum anotador, nenhum especialista, nenhum banco de partidas.
- ▶ **O adversário está sempre à altura.** Como é uma cópia de si mesmo, a taxa de vitórias fica em torno de 50% durante todo o treino.
- ▶ Um adversário na medida é a definição operacional de um **currículo**: a dificuldade sobe exatamente na velocidade da competência.

Teste sua compreensão

Em que isso ajuda o treino da política? E o que acontece com a densidade do sinal de recompensa? *Pergunta para discussão conduzida em aula.*

Resposta

Com jogadores equilibrados, z é quase uma moeda honesta: a entropia do resultado é máxima e cada partida carrega perto de 1 bit sobre θ . Contra um adversário muito mais forte, $z \equiv -1$ e não há nada a aprender; contra um muito mais fraco, o mesmo com o sinal trocado. O auto-jogo mantém a recompensa **densa em informação**.

Intuição

Isto é uma resposta parcial à pergunta “por que RL funciona em jogos e sofre no mundo real”. Jogos de dois jogadores vêm com um gerador de currículo embutido. Robótica não vem.

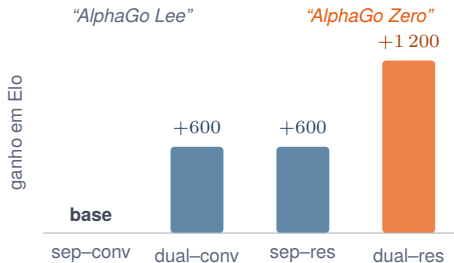
SEÇÃO 4

O que a evidência mostra

04

Quanto vem da arquitetura?

Quatro redes treinadas no *mesmo* conjunto fixo de partidas de auto-jogo, combinando: política e valor *separadas* (sep) ou *conjuntas* (dual); rede *convolucional* (conv) ou *residual* (res). Todas avaliadas com a mesma busca, 5 s por jogada.

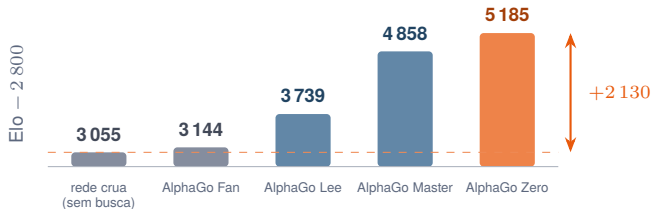


Intuição

Cada mudança de arquitetura vale, sozinha, mais de **600 Elo**; juntas, mais de 1 200. Resultado desconfortável: parte substancial do avanço atribuído ao “algoritmo” era engenharia de rede.

Quanto vem da busca?

A rede treinada do AlphaGo Zero, jogando **sem nenhum lookahead** — apenas $\arg \max_a p_\theta(s)_a$ — contra o sistema completo:



A busca vale $\approx 2\,100$ Elo

A mesma rede, com e sem MCTS, difere em mais de 2 000 pontos — numa escala em que 200 pontos já são 75% de vitórias.

Intuição

A busca é um **amplificador de computação em tempo de decisão** — a mesma alavanca hoje explorada em modelos de linguagem sob o nome de *test-time compute*.

Os dados humanos são necessários?

Comparação direta feita no artigo: o mesmo laço de treino, partindo de uma rede inicializada por aprendizado supervisionado sobre partidas humanas (KGS), versus partindo de pesos aleatórios.

- ▶ A versão supervisionada aprende **mais rápido no início** e prevê jogadas humanas com mais acurácia — o tempo todo.
- ▶ A versão *tabula rasa* ultrapassa a supervisionada em poucas horas e termina **mais forte**.
- ▶ AlphaGo Zero derrotou a versão que venceu Lee Sedol por **100 a 0**, usando 4 TPUs contra 48.
- ▶ Rodada de 3 dias: 4,9 milhões de partidas de auto-jogo. Rodada de 40 dias, rede de 40 blocos: 29 milhões de partidas, Elo 5 185.

Atenção

Prever bem o humano e jogar bem não são a mesma coisa. A rede supervisionada é melhor na primeira tarefa e pior na segunda. Imitação carrega o teto do imitado; o auto-jogo não tem esse teto.

Intuição

Cuidado ao generalizar: aqui há simulador perfeito e recompensa verificável. Onde faltam essas duas coisas — diálogo, medicina, direção — a conclusão “dados humanos são dispensáveis” *não* se transfere.

Um algoritmo, três jogos

O AlphaZero removeu o que restava de específico do Go (simetrias do tabuleiro, torneio de avaliação entre gerações) e rodou o mesmo código em xadrez, shogi e Go.

	AlphaZero	Stockfish 8
Paradigma de busca	MCTS guiada por rede	alfa-beta com poda
Posições por segundo	$\approx 6 \times 10^4$	$\approx 6 \times 10^7$
Conhecimento embutido	regras do xadrez	livro de aberturas, avaliação, finais
Partidas (Science 2018)	1 000 jogos: 155 vitórias, 6 derrotas, 839 empates	
Treino	700 000 passos; ultrapassou o Stockfish em ≈ 4 h	

Intuição

Mil vezes menos posições examinadas, e ainda assim vitória: a rede não acelera a busca, ela escolhe *muito melhor* o que examinar.

Atenção

As condições foram contestadas: versão do Stockfish, hardware, controle de tempo, ausência de livro de aberturas. Com desvantagem de tempo o AlphaZero se manteve superior até 10:1. A comparação é informativa, não uma medição limpa.

SEÇÃO 5

Além do tabuleiro

05

A ideia que sobrevive à mudança de domínio

A receita, sem menção a jogos

Se um problema pode ser escrito como *sequência de escolhas* com um simulador que verifica o resultado, então uma rede que propõe candidatos mais uma busca que testa candidatos pode superar a heurística humana — mesmo quando o espaço de busca é astronômico.

Go / xadrez

estado: tabuleiro

ação: jogada

prêmio: vitória

AlphaTensor

estado: tensor
residual

ação: tríade de rank 1

prêmio: rank baixo

AlphaDev

estado:

programa parcial

ação: instrução

prêmio: correto
e rápido

MuZero

estado: latente

ação: ação

modelo: **aprendido**

A única peça que muda é o simulador. A busca, o PUCT, o auto-jogo e a perda conjunta permanecem.

AlphaTensor: multiplicar matrizes mais rápido

Multiplicar duas matrizes $n \times n$ é equivalente a decompor um tensor $\mathcal{T}_n$ em somas de tríades de posto 1. O *posto* da decomposição é o número de multiplicações escalares. Cada ação do jogo é uma tríade; o episódio termina quando o tensor residual zera.

Problema	melhor conhecido	AlphaTensor	observação
2×2 (Strassen, 1969)	7	7	redescoberto do zero
3×3 (Laderman, 1976)	23	23	redescoberto do zero
4×4 sobre $\mathbb{Z}_2$	49	47	primeiro avanço desde 1969
4×5 por 5×5 (reais)	80	76	
5×5 (reais)	98	96	

Intuição

O resultado do 4×4 vale em aritmética modular ($\mathbb{Z}_2$), não nos reais — distinção que boa parte da imprensa perdeu. Ainda assim, é o primeiro avanço sobre Strassen em dois níveis em meio século.

Atenção

Penalizar o *tempo medido* numa GPU V100 rendeu algoritmos 10–20% mais rápidos *naquele* hardware: otimalidade é relativa à máquina. Meses depois, Kauers e Moosbauer baixaram o 5×5 sobre GF(2) a 95 por busca clássica.

AlphaDev: escrever assembly mais rápido que o humano

AssemblyGame: o estado é o programa em assembly construído até agora; a ação é acrescentar uma instrução; a recompensa combina *correção* (testada contra entradas) e *latência*.

- ▶ Alvo: `sort3`, `sort4`, `sort5` — rotinas chamadas por dentro de qualquer ordenação maior.
- ▶ Descobriu duas sequências novas, batizadas de *swap* e *copy* do AlphaDev, que economizam uma instrução cada vez que são aplicadas.
- ▶ `VarSort4`: algoritmo com estrutura diferente, 29 instruções mais curto que o de referência humano.
- ▶ Integrado à `libc++` do LLVM — a primeira mudança nessa parte da biblioteca em mais de uma década, e a primeira originada de aprendizagem por reforço.

Ganhos medidos

Até **70%** mais rápido em sequências de cinco elementos; $\approx 1,7\%$ em sequências acima de 250 000 elementos. Também $\approx 30\%$ em funções de *hash* na faixa de 9 a 16 bytes.

Intuição

Por que 70% num caso e 1,7% no outro? Porque as rotinas otimizadas são as *folhas* de algoritmos maiores. Melhorar a folha melhora muito a folha e pouco o todo — mas a folha é chamada trilhões de vezes por dia.

MuZero e o retorno do modelo desconhecido

AlphaZero exige um simulador perfeito. O MuZero remove essa exigência: em vez de usar as regras, **aprende** um modelo — mas apenas o suficiente para planejar.

Três funções aprendidas conjuntamente:

- ▶ **Representação** h : observação $\mapsto$ estado latente s^0 .
- ▶ **Dinâmica** g : $(s^k, a) \mapsto (s^{k+1}, r^k)$, no espaço latente.
- ▶ **Predição** f : $s^k \mapsto (p, v)$.

A MCTS roda *inteiramente* no espaço latente. Nada obriga s^k a reconstruir a observação — o modelo é treinado só para que política, valor e recompensa saiam certos.

Intuição

Modelo suficiente para decidir, não para prever. É a diferença entre um modelo *gerativo* do mundo e um modelo *orientado ao valor*. Prever pixels é caro e, em boa parte, irrelevante para a decisão.

Definição — Alcance

Mesmo algoritmo, mesmo desempenho em Go, xadrez e shogi — e estado da arte em Atari, onde as regras não são dadas.

SEÇÃO 6

Um olhar crítico: o UCT é o algoritmo certo?

06

A pergunta incômoda

O UCT trata cada nó como um bandit e usa uma cota superior de confiança para escolher a ação a simular.

Teste sua compreensão

Por que isso é um pouco estranho? Lembre por que as cotas superiores de confiança eram uma boa ideia em bandits. Existe mesmo um problema de exploração *versus* aproveitamento **dentro** de episódios simulados? *Pergunta para discussão conduzida em aula.*

A pergunta incômoda

O UCT trata cada nó como um bandit e usa uma cota superior de confiança para escolher a ação a simular.

Teste sua compreensão

Por que isso é um pouco estranho? Lembre por que as cotas superiores de confiança eram uma boa ideia em bandits. Existe mesmo um problema de exploração *versus* aproveitamento **dentro** de episódios simulados? *Pergunta para discussão conduzida em aula.*

Resposta

Em um bandit real, cada braço puxado *custa*: a recompensa perdida é perda de verdade, e minimizar arrependimento cumulativo é o objetivo correto. Dentro da simulação não se perde nada. Puxar mil vezes um braço ruim custa apenas tempo de CPU. O objetivo aqui não é acumular recompensa simulada — é **tomar a melhor decisão na raiz ao final do orçamento**. São dois problemas diferentes, e o UCT resolve o errado.

Arrependimento cumulativo e arrependimento simples

Definição — Arrependimento cumulativo

$$R_n = \sum_{t=1}^n (\mu^* - \mu_{A_t})$$

O que se *pagou* ao longo do caminho. É o objetivo quando as ações são reais.

Definição — Arrependimento simples

$$r_n = \mu^* - \mu_{\hat{A}_n}$$

A qualidade da *recomendação final* $\hat{A}_n$. É o objetivo quando as ações são simuladas.

Bubeck, Munos e Stoltz (2009): há um conflito

Quanto mais um algoritmo minimiza o arrependimento *cumulativo*, pior fica seu arrependimento *simples* no pior caso. Alocação uniforme: $r_n = O(e^{-cn})$.

Algoritmo com $R_n = O(\log n)$, como o UCB: existe instância em que r_n decai apenas **polinomialmente** em n .

Intuição

A intuição é direta: para ter R_n pequeno, é preciso parar de amostrar os braços aparentemente ruins — e é exatamente amostrá-los que traria certeza sobre qual é o melhor.

O que se deveria fazer: raciocínio metanível

A pergunta correta não é “qual ação simular para ganhar mais recompensa simulada”, e sim:

O problema metanível

Qual é a próxima computação a executar, dado que ela custa tempo e que o benefício é a chance de mudar a decisão da raiz?

- ▶ Isto é, literalmente, um MDP — cujos estados são *estados de crença sobre valores* e cujas ações são *computações* (Hay, Russell, Tolpin e Shimony, 2012).
- ▶ Uma computação só vale a pena se puder **mudar** a jogada escolhida. Refinar a estimativa do braço que já está em último lugar é desperdício, por mais incerta que ela seja.
- ▶ Algoritmos desenhados para esse objetivo — BRUE, SHOT, e variantes de *sequential halving* em árvores — superam o UCT em vários cenários de orçamento fixo.

Intuição

Curiosamente, o **PUCT se afasta do UCT na direção certa**, ainda que por acidente histórico: o decaimento $N(s, a)^{-1}$ e o crescimento $\sqrt{N(s)}$ distribuem visitas mais como uma *política* do que como uma cota de confiança — que é o que Grill et al. formalizam.

A parede da profundidade

Um experimento pequeno, um resultado desconfortável

Árvore-armadilha (no espírito de Coquelin e Munos, 2007). Em cada profundidade d : *parar* termina com valor $0,9 - 0,4 d/D$; *seguir* até $d = D$ vale 1,0, o ótimo. A tentação está no começo e a recompensa está no fim.

Prob. de a raiz escolher a ação ótima
(MCTS com rollout aleatória; melhor c por célula)

D	$K = 200$	1 000	5 000	20 000
4	0,85	1,00	1,00	1,00
6	0,00	0,20	1,00	1,00
8	0,00	0,00	0,00	0,00
10	0,00	0,00	0,00	0,00

Com $D = 10$, nem 50 000 bastam.

Atenção

Duas causas. (i) A rollout aleatória alcança a folha profunda com probabilidade 2^{-D} : o valor bom quase nunca é visto. (ii) O UCT retropropaga a *média*, não o máximo — e a política que a gerou foi mudando, então o viés de cada nível se compõe ao subir.

Intuição

Coquelin e Munos exibem árvores em que o UCT precisa de um número de amostras *exponencial em uma exponencial* na profundidade. A consistência assintótica é verdadeira e inútil.

O ajuste de c não é secundário

Mesma árvore com $D = 6$ e orçamento fixo de 2 000 simulações, variando apenas a constante de exploração:



Atenção

A curva **não é monotônica nem suave**: $c = 0,7$ acerta 65% das vezes e $c = 1,0$ falha sempre. Pouca exploração nunca sai da armadilha rasa; muita contamina as médias retropropagadas.

Intuição

Isso explica os 94 e 103 pontos de Elo obtidos apenas com otimização bayesiana dos hiperparâmetros da busca no AlphaGo. Ao ler um resultado de MCTS, pergunte sempre *quanto* de ajuste houve — a diferença entre um valor de c e outro pode ser maior que a diferença entre dois algoritmos.

Então por que a MCTS funciona tão bem na prática?

1. **A rollout foi eliminada.** A causa (i) da parede da profundidade desaparece quando a folha é avaliada por v_θ em vez de por uma partida aleatória. O bom valor não precisa mais ser *encontrado* por sorte.
2. **O prior poda a largura.** Das ≈ 250 jogadas legais no Go, o PUCT examina seriamente uma dúzia. O fator de ramificação efetivo despenca.
3. **As árvores reais não são adversárias.** Nas armadilhas construídas por Coquelin e Munos o valor está escondido atrás de um caminho estreito; em Go e xadrez, posições boas costumam ter vizinhos bons — o gradiente existe.
4. **O laço de treino conserta a heurística.** Onde v_θ erra sistematicamente, o auto-jogo gera dados exatamente ali.

A leitura para levar

A MCTS moderna funciona apesar de o UCT ser questionável, não por causa dele: o peso saiu da regra de seleção e foi para a **qualidade do avaliador de folha**.

Atenção

Consequência prática: ao diagnosticar um planejador que decide mal, suspeite primeiro do avaliador de folha e do prior, e só depois da constante de exploração ou do orçamento.

SEÇÃO 7

Testes de compreensão

07

Quando a MCTS é uma boa escolha?

Teste sua compreensão

A MCTS decide qual ação tomar por busca em árvore, escolhendo ações que maximizam $Q(s, a)$ e *amostrando* sucessores. Em quais situações ela é boa escolha?

1. Horizonte curto, poucos estados e poucas ações.
2. Horizonte longo, **muitas ações** e poucos estados.
3. Horizonte longo, **muitos estados** e poucas ações.

Pergunta para discussão conduzida em aula.

Quando a MCTS é uma boa escolha?

Teste sua compreensão

A MCTS decide qual ação tomar por busca em árvore, escolhendo ações que maximizam $Q(s, a)$ e *amostrando* sucessores. Em quais situações ela é boa escolha?

1. Horizonte curto, poucos estados e poucas ações.
2. Horizonte longo, **muitas ações** e poucos estados.
3. Horizonte longo, **muitos estados** e poucas ações.

Pergunta para discussão conduzida em aula.

Resposta

Falso, falso, verdadeiro. **(1)** Poucos estados e horizonte curto: resolva o MDP exatamente — a iteração de valor devolve π^* para *todos* os estados pelo mesmo preço. **(2)** A árvore ramifica *em cada ação*, e com poucos estados a programação dinâmica volta a ser viável. **(3)** Aqui a amostragem paga: substitui a soma sobre s' e faz o custo depender do orçamento, não de $|\mathcal{S}|$.

Tratar cada nó como um bandit

Teste sua compreensão

No UCT, cada nó é tratado como um bandit e uma cota superior de confiança sobre o valor futuro de cada ação decide o que simular em seguida. Quais afirmações são verdadeiras?

1. É útil porque prioriza ações que levam a boas recompensas adiante.
2. O UCB minimiza arrependimento; o UCT minimiza arrependimento *dentro* das rollouts da árvore — e, se é verdade, é uma boa ideia?

Pergunta para discussão conduzida em aula.

Tratar cada nó como um bandit

Teste sua compreensão

No UCT, cada nó é tratado como um bandit e uma cota superior de confiança sobre o valor futuro de cada ação decide o que simular em seguida. Quais afirmações são verdadeiras?

1. É útil porque prioriza ações que levam a boas recompensas adiante.
2. O UCB minimiza arrependimento; o UCT minimiza arrependimento *dentro* das rollouts da árvore — e, se é verdade, é uma boa ideia?

Pergunta para discussão conduzida em aula.

Resposta

As duas são verdadeiras — e a segunda é o ponto da aula. O UCT de fato minimiza arrependimento cumulativo ao longo dos episódios simulados, mas arrependimento simulado **não é um custo**: nada se perde ao explorar uma linha ruim dentro do simulador. O objetivo correto é o arrependimento *simples* na raiz — e minimizar um prejudica o outro.

Revisão: para que servem as cotas superiores de confiança

Teste sua compreensão

Quais afirmações são verdadeiras?

1. Cotas superiores de confiança equilibram exploração e aproveitamento da informação já obtida.
2. Servem tanto em bandits quanto em processos de decisão de Markov.
3. Se o modelo de recompensa é conhecido, não há benefício em usá-las.

Pergunta para discussão conduzida em aula.

Revisão: para que servem as cotas superiores de confiança

Teste sua compreensão

Quais afirmações são verdadeiras?

1. Cotas superiores de confiança equilibram exploração e aproveitamento da informação já obtida.
2. Servem tanto em bandits quanto em processos de decisão de Markov.
3. Se o modelo de recompensa é conhecido, não há benefício em usá-las.

Pergunta para discussão conduzida em aula.

Resposta

Verdadeira. Verdadeira. Depende do cenário. Em um **bandit**, conhecer o modelo de recompensa encerra o problema. Em um **MDP**, não: mesmo com R conhecida, se $P(s' | s, a)$ é desconhecida ainda há incerteza sobre *para onde* cada ação leva — recompensa conhecida não implica valor conhecido.

SEÇÃO 8

Síntese

08

O que levar desta aula

1. **Go não era um problema de modelo desconhecido.** Era um problema de modelo conhecido e grande demais para usar. AlphaZero é planejamento com heurísticas aprendidas.
2. **A busca é um operador de melhoria de política.** Treinar a rede para imitar a saída da busca é iteração de política com a busca no lugar do passo guloso.
3. **Auto-jogo é um gerador de currículo.** O adversário sempre à altura mantém o resultado incerto e, portanto, informativo.
4. **A alavanca principal é o avaliador de folha.** Trocar rollout por rede de valor é a mesma troca de Monte Carlo por TD, com o mesmo balanço viés–variância.
5. **O PUCT não é uma cota de confiança.** É uma regularização implícita em direção ao prior, que enfraquece com $N^{-1/2}$.
6. **O UCT resolve o problema errado.** Dentro da simulação, o que importa é o arrependimento simples, não o cumulativo. A MCTS moderna funciona apesar disso.
7. **Busca é computação em tempo de decisão.** A mesma rede vale 2 100 Elo a mais quando pensa antes de responder.
8. **Cuidado ao extrapolar.** Simulador perfeito mais recompensa verificável é uma combinação rara. Sem ela, “dispensar dados humanos” não se transfere.

Fontes primárias

- ▶ Silver et al., *Mastering the game of Go with deep neural networks and tree search*, Nature 529, 2016. (AlphaGo; a mistura rollout/valor.)
- ▶ Silver et al., *Mastering the game of Go without human knowledge*, Nature 550, 2017. (AlphaGo Zero; as ablações de arquitetura e busca.)
- ▶ Silver et al., *A general reinforcement learning algorithm that masters chess, shogi and Go through self-play*, Science 362, 2018.
- ▶ Schrittwieser et al., *Mastering Atari, Go, chess and shogi by planning with a learned model*, Nature 588, 2020. (MuZero.)
- ▶ Fawzi et al., *Discovering faster matrix multiplication algorithms with reinforcement learning*, Nature 610, 2022. (AlphaTensor.)
- ▶ Mankowitz et al., *Faster sorting algorithms discovered using deep reinforcement learning*, Nature 618, 2023. (AlphaDev.)

Fundamentos e crítica

- ▶ Kocsis e Szepesvári, *Bandit based Monte-Carlo planning*, ECML 2006. (UCT.)
- ▶ Coquelin e Munos, *Bandit algorithms for tree search*, UAI 2007. (As patologias.)
- ▶ Bubeck, Munos e Stoltz, *Pure exploration in multi-armed bandits problems*, ALT 2009. (Arrependimento simples versus cumulativo.)
- ▶ Hay, Russell, Tolpin e Shimony, *Selecting computations: theory and applications*, UAI 2012. (Raciocínio metanível.)
- ▶ Anthony, Tian e Barber, *Thinking fast and slow with deep learning and tree search*, NeurIPS 2017. (*Expert Iteration*.)
- ▶ Grill et al., *Monte-Carlo tree search as regularized policy optimization*, ICML 2020.
- ▶ Chen et al., *Bayesian optimization in AlphaGo*, arXiv:1812.06855, 2018.
- ▶ Browne et al., *A survey of Monte Carlo tree search methods*, IEEE TCIAIG 4(1), 2012. (Panorama.)